UNIFY

Dataset Construction and Validation Action Plan

Student-facing implementation guide

Prepared from the latest post-defence proposal and the panel requirement to validate the project datasets.

Project	Information
Project	UNIFY — Cross-Tool Change Assurance for Product Teams
Students	Abdullah Amin · Haaniah Ismail · Mishal Fatima
Supervisor	Dr. Ali Zeeshan Ijaz
Purpose	Convert the proposal's dataset plan into a reproducible, independently checked evaluation asset.
Date	6 October 2026

Central instruction

The panel comment is not satisfied merely by naming public datasets or writing a train/validation/test split. UNIFY must demonstrate that its repositories, labels, evidence spans, verdicts and splits are trustworthy, reproducible and appropriate for the exact TypeScript/React problem being claimed.

Required outcome

Produce a versioned "UNIFY Dataset Package v1" containing the data, annotation manual, independent labels, adjudication record, split manifest, automated integrity report and dataset card.

1. Executive direction

The latest proposal has a strong conceptual response to the panel: it defines a custom Change-Intent Oracle, repository-level splitting, hard negatives, abstention cases, label agreement, integrity checks and a fine-tuning go/no-go rule. The remaining task is to execute and consolidate that plan.

Status	Assessment
Already appropriate	Custom oracle; independent labelling; adjudication; ambiguous cases; hard negatives; repository-level splits; dataset card; integrity checks; untouched test project.
Must be corrected	TypeScript/React evaluation cannot primarily rely on Python corpora; verdict labels are inconsistent; student-only adjudication needs external audit; Sections 12.3 and 28 overlap.
Must now be produced	Repository catalogue, case files, annotation guide, agreement results, audit trail, split manifest, integrity/leakage report and frozen gold test set.

What dataset validation means for UNIFY

- The selected repositories match the declared TypeScript/React repository class.
- Every requirement-to-code/test relationship and final verdict is supported by exact evidence.
- Different annotators can apply the label definitions consistently.
- Training, validation and test projects do not leak into one another.
- Synthetic cases are identified and do not dominate the final conclusion.
- Every commit, file, span, test and source version can be reproduced.
- The final gold labels are human-established and independently audited.

> **Important distinction**
>
> Dataset validation establishes whether the ground truth is trustworthy. Model validation measures performance against that ground truth. A high F1 score on a leaked or incorrectly labelled dataset is not credible evidence.

Required proposal corrections before annotation begins

Issue	Required correction
Language mismatch	Use TypeScript/React repositories for the main gold evaluation. Treat Python corpora only as supplementary training, schema or cross-language evidence.
Verdict inconsistency	Use four verdicts only. Record stale data and human review as separate fields.
Circular validation	A student must not validate a case they constructed. Add supervisor and external audit of the gold set.
Duplicated specification	Merge proposal Sections 12.3 and 28 into one authoritative dataset protocol.

Issue	Required correction
Unfrozen splits	Allocate repositories to train, development and test before model or prompt experimentation.

2. Freeze the label taxonomy first

The current proposal names SOURCE_STALE alongside verdicts and also refers to REVIEW, although the output schema defines only four statuses. Do not label any full dataset until this is resolved.

Field	Allowed values	Purpose
verdict	SUPPORTED · CONTRADICTED · PARTIAL · INSUFFICIENT_EVIDENCE	The class used for model evaluation.
freshness_state	CURRENT · STALE · UNKNOWN	Whether tool evidence establishes the current state.
review_required	true · false	Whether a person must inspect the case.
reason_code	SOURCE_STALE · MISSING_TEST · AMBIGUOUS_LINK · INVALID_CITATION · …	Explains abstention, downgrade or review.
origin	REAL · CONTROLLED_MUTATION · SYNTHETIC	Keeps constructed evidence separate from real evidence.

> **Example**
>
> If a Jira or Figma snapshot is stale: verdict = INSUFFICIENT_EVIDENCE; freshness_state = STALE; reason_code = SOURCE_STALE; review_required = true.

3. Dataset architecture

Asset	Purpose	Recommended scale
Repository eligibility catalogue	Records every candidate repository and why it was accepted or rejected.	12–20 candidates
Training pool	PEFT and retrieval training; may include controlled transformations.	200–500 examples
Development/validation set	Model selection, prompt development and threshold setting.	20–30 fully labelled cases
Frozen gold test oracle	Final unbiased evaluation of links, retrieval, verdicts and abstention.	30–50 fully adjudicated cases
Event-reliability suite	Duplicate, delayed, missing, replayed and out-of-order event tests.	20–30 controlled sequences
User-acceptance task set	Manual workflow versus UNIFY evidence-case comparison.	6–10 realistic tasks

Repository-level allocation

- Training/development: approximately 8–12 repositories.
- Validation: 2–3 repositories not used for training.
- Final testing: 2–3 completely unseen repositories.
- At least one final-test repository should remain unopened for experimental analysis until the pipeline and thresholds are frozen.
- Forks, copied templates and near-duplicate projects must remain in the same split.

4. Repository eligibility and public-dataset use

4.1 Mandatory repository gate

Check	Acceptance condition
Language and architecture	TypeScript or JavaScript/React is the principal relevant language and the repository matches the committed analyser.
Licence	A recorded, compatible open-source licence permits the intended research use.
Reproducibility	Base/head commits resolve; the project installs; relevant tests can be run or their failure is documented.
Intent evidence	Usable issue/PR history exists and at least one atomic criterion can be derived without inventing intent.
Code/test evidence	Changed files and symbols can be extracted; test evidence can be located or explicitly marked missing.
Data hygiene	Generated/vendor files are identifiable; no secrets, tokens or private user information are retained.
Provenance	Every retained artifact has a source, version, retrieval date and checksum.

4.2 Documentation-quality cohorts

Because names, comments and directory structures may make retrieval artificially easy, profile each repository and report performance separately for well-, moderately- and poorly documented projects.

- Comment and docstring coverage of public symbols.
- Identifier informativeness and cryptic-name rate.
- Directory-structure conventionality.
- README, module or architectural documentation.
- Test presence, discoverability and naming quality.

4.3 Correct use of public corpora

Source	Appropriate use	Restriction
SWE-bench Verified	Issue–patch–test schema experiments, supplementary training or baseline work.	Its core repositories are Python; it cannot validate the committed TypeScript/React analyser.

Source	Appropriate use	Restriction
PyBugHive	Reproducible bug/patch/test structure and evaluation-design reference.	It is a Python benchmark and cannot establish Figma, Jira or TypeScript performance.
SWE-bench Multimodal	Supplementary front-end/visual issue cases and UI-oriented evidence.	Screenshots and UI issues are not direct Figma-node or approved-design-constraint ground truth.
UNIFY fixture repositories	Exact Figma/Jira links, structured constraints, event order, stale-source and ambiguity cases.	Must be clearly marked as controlled and cannot alone establish real-world diversity.

Decision

The final headline results must come from the TypeScript/React UNIFY oracle. Python-based results may be reported separately, but must not be blended into the main score.

5. Structure of one dataset case

Field group	Required contents
Identity	Case ID, dataset version, origin and difficulty.
Repository	URL, licence, repository ID and eligibility record.
Versions	Base SHA, head SHA and environment/tool versions.
Intent	One atomic Jira-style criterion and its source/version.
Design evidence	Figma file/node/version or approved structured constraint.
Code evidence	Expected changed files, symbols, dependencies and exact line/diff spans.
Test evidence	Relevant test IDs, changed tests, coverage where available, and execution result.
Trace anchors	Requirement-to-symbol/test relationships with provenance.
Events	Relevant Jira, GitHub and Figma events in their delivered order.
Gold outcome	Verdict, freshness state, reason code and review requirement.
Justification	Exact evidence spans and a written reason for the label.
Annotation record	Two original labels, disagreement notes, adjudicated label and auditor decision.

6. How cases should be constructed

Case type	Construction method
Real supported/contradicted cases	Start from a real issue and PR. Atomise the criterion, pin base/head commits, inspect changed symbols and tests, and label from evidence—not merely from merge status.
Hard negatives	Use unrelated but plausible files/symbols from the same repository and preferably the same module. Similar names and vocabulary should make the case genuinely difficult.
Controlled contradictions	Change a criterion without code; alter code without linked intent; change a design constraint after branch creation; remove a required test; modify a similar but wrong component.
Ambiguous cases	Use missing tests, stale sources, unresolved links, incomplete criteria, unsupported dynamic behaviour or deliberately truncated evidence. Correct behaviour is usually abstention.
Event-reliability cases	Deliver duplicates, reordered events, delays, missed webhooks, API outages, stale checkpoints and replayed events; predeclare the expected final graph state.
Low-documentation cases	Retain eligible projects with weak names/comments and report them as a separate cohort rather than removing all difficult repositories.

> **Gold-label rule**
>
> A decisive label without an exact cited criterion and implementation/test evidence is not a valid gold label. A merged pull request is evidence of acceptance, not automatic proof of implementation correctness.

7. Who constructs and validates the data?

Role	Responsibility	Restriction
Case constructor	Collects artifacts, records provenance, checks build/tests and creates the case packet.	Must not be one of the two primary annotators for that case.
Annotators A and B	Independently label links, verdict, evidence spans, freshness and missing evidence.	Must work blind and submit before discussion.
Adjudicator	Reviews disagreements and records the final decision and rationale.	For gold-test disagreements, involve the supervisor or appointed independent reviewer.
Supervisor	Approves taxonomy/manual, audits 10–20% of gold cases, reviews unresolved disagreements and approves/freeze splits.	Need not label the entire training pool.
External reviewer	Audits a small representative sample and critical edge cases.	Should be a developer/reviewer; use a designer/product participant for structured design constraints where possible.

Role	Responsibility	Restriction
Automated validator	Checks SHAs, files, spans, test IDs, completeness, secrets, duplicates, checksums and split leakage.	Automation validates integrity, not semantic correctness of labels.

Recommended student ownership

Student	Primary dataset responsibility
Haaniah Ismail	Repository/artifact ingestion, event sequences, provenance, licences and integrity manifest.
Mishal Fatima	Code/test anchors, evidence-span extraction, hard-negative mining and anchor validation.
Abdullah Amin	Annotation format/tooling, split management, leakage checks, dataset card and evaluation scripts.

All three students participate in blind annotation through rotating case batches. No student should be the sole validator of data produced by their own subsystem.

Independent audit minimum

- A random 10–20% sample of the frozen gold set.
- Every rare verdict class and every case used as a defence demonstration.
- All high-impact contradictions and all unresolved annotation disagreements.
- Several ambiguous cases where abstention is the expected outcome.

8. Annotation and adjudication process

1. **Freeze the manual** — Define atomic criteria, link eligibility, four verdicts, PARTIAL versus INSUFFICIENT_EVIDENCE, evidence spans, freshness, missing evidence and worked examples.
2. **Run a pilot** — Use 10–15 diverse cases before full labelling. Include hard negatives and ambiguous cases.
3. **Measure agreement** — Report raw agreement and Cohen's kappa for categorical labels. Measure evidence-span overlap separately using line/token F1 or intersection-over-union.
4. **Revise if necessary** — Target Cohen's kappa ≥ 0.70. If lower, analyse disagreements, rewrite the guide and repeat the pilot.
5. **Blind full annotation** — Annotators work independently; identities, timestamps and manual version are recorded.
6. **Adjudicate visibly** — Retain original labels. Record the final decision, the evidence used and the rule added or clarified.
7. **Audit independently** — Supervisor/external reviewer checks the specified sample and all critical disagreements.
8. **Freeze and version** — Publish the dataset card, split manifest, integrity report and checksums before final model evaluation.

Agreement and quality metrics

Item	Measure / gate
Verdict and link labels	Raw agreement plus Cohen's kappa; target ≥ 0.70 after guide revision.
Evidence spans	Line/token overlap F1 or intersection-over-union; report exact-match rate as supplementary.
Integrity	100% resolvable commits, files, spans and mandatory fields.
Provenance/licence	100% of retained repositories and cases have source and licence records.
Cross-split duplication	Zero duplicate repositories, forks, copied templates or derived mutations across splits.
Class coverage	Enough examples per verdict to report per-class precision/recall; do not hide minority classes in one aggregate score.
Ambiguity	Explicit ambiguous subset; correct abstention measured separately.

9. Leakage, integrity and test-set controls

Hard leakage rules

- Split by repository, never by individual requirement–code pair.
- Keep forks, templates and copied components in the same split.
- A controlled mutation inherits the split of its source repository.
- Do not use test cases in prompts, few-shot examples, threshold selection or model choice.
- Generate synthetic training examples only from training repositories.
- Deduplicate issue text, patches and code fingerprints across splits.
- Freeze the split manifest before fine-tuning or extensive prompt experimentation.
- Open the final test labels only after the model, prompt, retrieval method and thresholds are frozen.

Automated validation report

Check	Pass condition
Commit resolution	Every base/head SHA can be checked out.
Anchor integrity	Every file and line/span reference exists at the recorded commit.
Test identity	Every cited test exists and its recorded result is reproducible or explicitly unavailable.
Packet completeness	Every mandatory schema field is present and type-valid.
Citation validity	Every decisive finding cites an artifact inside the immutable packet.
Secret/privacy scan	No token, credential or private participant information remains.

Check	Pass condition
Duplicate detection	No duplicate/near-duplicate crosses repository splits.
Checksum manifest	Every frozen case and source export has a recorded checksum.

Test-set governance

The supervisor should retain the frozen test labels, or otherwise control their release, until the students declare the final model, adapter, prompt, retrieval configuration and thresholds. The final evaluation should then be executed from a tagged code version with recorded hardware and configuration.

> **Why this matters**
>
> Repository-level leakage is the easiest way for UNIFY to learn project-specific names and directory patterns and then report an impressive but meaningless result on “unseen” pairs from the same project.

10. Immediate implementation plan

Phase	Student actions	Required evidence
Phase 1 — Protocol freeze	Merge Sections 12.3 and 28; freeze taxonomy, schema, eligibility gate, roles and split policy.	Dataset Protocol v1 and revised proposal text.
Phase 2 — Repository screening	Inspect 12–20 candidates; record language, licence, history, testability, documentation quality and acceptance/rejection.	Repository eligibility catalogue.
Phase 3 — Pilot	Construct 12–20 cases from at least three TypeScript/React repositories; independently label 10–15.	Case packets, annotation files, agreement and disagreement report.
Phase 4 — Guide revision	Revise definitions until verdict/link kappa reaches the declared gate; freeze annotation manual v1.	Versioned guide and second pilot result if required.
Phase 5 — Full construction	Build the training pool, development set, gold oracle and event suite; preserve real/synthetic origin.	Versioned dataset with provenance.
Phase 6 — Audit and freeze	Run automated checks, supervisor/external audit, deduplication and repository-level split verification.	Dataset card, integrity report, audit log, split and checksum manifests.
Phase 7 — Evaluation	Freeze model/prompt/retrieval; evaluate once on gold test; report per-class results and error analysis.	Reproducible evaluation report and experiment logs.

Minimum evidence for the supervisor mid-evaluation

1. Frozen four-verdict taxonomy and freshness/review fields.
2. Repository eligibility checklist and at least three accepted TypeScript/React repositories.
3. A pilot set of approximately 12–20 cases covering supported, contradicted, partial and insufficient-evidence outcomes.

4. Several same-module hard negatives and at least three deliberate abstention cases.
5. Written annotation manual with worked examples.
6. Two independent annotation files and preliminary agreement statistics.
7. At least one fully documented adjudication.
8. Dataset card draft, split plan and automated integrity-check output.
9. One end-to-end case demonstrated from source artifacts to gold verdict.

> **What is not required immediately**
>
> The complete 200–500-example training pool and 30–50-case frozen gold test set do not need to be finished for the first supervisor checkpoint. The protocol and pilot must be credible first; scaling an invalid labelling process would only create more rework.

11. Final dataset package and defence checklist

Deliverable	Contents
01_repository_catalogue.csv	Candidate repositories, language, licence, testability, documentation profile, decision and reason.
02_case_schema.json	Machine-readable schema and allowed values.
03_annotation_manual.pdf	Definitions, decision rules and worked examples.
04_cases/	Versioned case packets and immutable evidence.
05_raw_annotations/	Independent labels preserved unchanged.
06_adjudication_log.csv	Disagreements, decisions, rationale and adjudicator.
07_split_manifest.csv	Repository-level train/development/test assignment.
08_integrity_report.html/pdf	Commit, span, citation, duplicate, secret and completeness checks.
09_dataset_card.pdf	Provenance, licences, size, balance, construction, validation and limitations.
10_checksums.txt	Hashes of frozen cases and manifests.
11_evaluation_config/	Pinned model, adapter, prompt, retrieval, thresholds, hardware and code tag.
12_results/	Per-class metrics, confusion matrix, abstention, error analysis and audit record.

Questions the students must be able to answer

- Why is this repository eligible, and why were other repositories rejected?
- Who produced this gold label, who disagreed, and who adjudicated it?
- What exact evidence supports this SUPPORTED or CONTRADICTED outcome?
- How do PARTIAL and INSUFFICIENT_EVIDENCE differ in the annotation manual?
- How was repository-level leakage prevented?
- How much of the training pool is real, controlled or synthetic?
- What happens to results when comments, names or directory signals are removed?

- Which cases were independently audited?
- When was the test set opened, and what was frozen before that?
- What limitations prevent the results from being generalised beyond the declared repository class?

> **Defence-ready statement**
>
> UNIFY was not merely tested on a collection of examples. Its ground truth was constructed transparently, independently checked, leakage-controlled, reproducible and matched to the exact TypeScript/React change-assurance problem the system claims to solve.

12. References and source notes

The project-specific recommendations in this guide are based on UNIFY_final_proposal_v2_post_defense.docx, particularly Sections 11.6–12.4 and 28–29. Public sources below are included only to clarify the role and limitations of component corpora.

SWE-bench official dataset documentation

SWE-bench Multimodal official overview

PyBugHive official repository

END OF GUIDE